

FC7xxx SPI Integration Manual

Rev.0.6

Revision History

Revision	Date	Changes
0.1	2023/07/14	Initial release for MCAL V0.1.0
0.3	2023/10/20	Updated for MCAL V0.3.0
0.4	2023/11/22	Updated for MCAL V0.4.0
0.6	2023/03/29	Updated for MCAL V0.6.0 - Added support for FC7240

Table of Contents

Revision History	2
Table of Contents	3
Chapter 1 Introduction	4
1.1 Introduction	4
Chapter 2 Building	5
2.1 Dependencies on Other Modules	5
2.2 Files Required for Compile	5
2.3 Add Plug-ins	6
Chapter 3 Memory	7
3.1 Sections in Memory Map	7
Chapter 4 Exclusive Area	9
Chapter 5 Interrupt Service Routine (ISR)	10
Chapter 6 Error Report	11
6.1 Det	11
6.2 Dem	12
Chapter 7 Function Calls	13
7.1 Function Calls during Startup	13
7.2 Function Calls during Shutdown	13
7.3 Function Calls during Wake-up	13
7.4 Function Calls during Runtime	13
Chapter 8 Other Requirements	14
8.1 Notification, Callback, Callout	14
8.2 Macros	14
Chapter 9 Integration Steps	15

Chapter 1 Introduction

1.1 Introduction

This integration manual describes the integration requirements for the SPI module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- Mcu: This module provides the clock reference point for SPI module.
- Port: This module provides the port pin configurations relevant to SPI buses.
- Common: This module is the basic module which used to choose the chip.
- Dem: This module is used for reporting production relevant error status.

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).
- Det: This module is necessary for enabling Development error detection.

Module initialization dependency

- Mcu: To use the SPI module, the user needs to initialize MCU module first.
- Port: To use the SPI module, SCK, PCSx, SIN, SOUT pins for each SPI should be configured.

2.2 Files Required for Compile

SPI module files:

- _MCAL/Src/Spi/Src/Spi.c
- _MCAL/Src/Spi/Src/Spi_FCSpi.c
- _MCAL/Src/Spi/Src/Spi_FCSpi_Irq.c
- _MCAL/Src/Spi/include/Spi.h
- _MCAL/Src/Spi/include/Spi_FCSpi.h
- _MCAL/Src/Spi/include/Spi_MemMap.h
- _MCAL/Src/Spi/include/Spi_RegOps.h
- _MCAL/Src/Spi/include/Spi_Version.h
- _MCAL_XXXX_generateXXXX/src/Spi_PBcfg.c
- _MCAL_XXXX_generateXXXX /src/Spi_Cfg.c
- _MCAL_XXXX_generateXXXX /include/Spi_Cfg.h

Common module files:

- _MCAL/Src/Common/include/Mcal.h
- _MCAL/Src/Common/include/Std_Types.h
- _MCAL/Src/Common/include/Platform_Types.h
- _MCAL/Src/Common/include/Compiler.h
- _MCAL/Src/Common/include/Compiler_Cfg.h
- _MCAL/Src/Common/include/CompilerDefinition.h

Det module files:

- Det.h

Rte module files:

- SchM_Spi.h

Dem module files:

- Dem.h

2.3 Add Plug-ins

SPI module plug-ins are developed for EB tresos Studio, so, to use SPI plug-ins on the EB tresos Studio, the user needs to add the SPI module to the EB plug-ins folder first.

- 1) Copy the SPI module(_MCAL/EB_Plugins/eclipse/plugins/Spi) folder to EB tresos plug-ins (EB/tresos/plugins/) folder.
- 2) Set the SPI module output location folder for the generated source file and header files.
- 3) Use EB tresos to configure the SPI module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
SPI_START_SEC_CONFIG_DATA_8 SPI_STOP_SEC_CONFIG_DATA_8 SPI_START_SEC_CONFIG_DATA_16 SPI_STOP_SEC_CONFIG_DATA_16 SPI_START_SEC_CONFIG_DATA_32 SPI_STOP_SEC_CONFIG_DATA_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (data).
SPI_START_SEC_CONFIG_DATA_UNSPECIFIED SPI_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start and stop of Memory Section for Config Data.
SPI_START_SEC_CONST_BOOLEAN SPI_STOP_SEC_CONST_BOOLEAN SPI_START_SEC_CONST_8 SPI_STOP_SEC_CONST_8 SPI_START_SEC_CONST_16 SPI_STOP_SEC_CONST_16 SPI_START_SEC_CONST_32 SPI_STOP_SEC_CONST_32 SPI_START_SEC_CONST_UNSPECIFIED SPI_STOP_SEC_CONST_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read only (rodata).
SPI_START_SEC_CODE SPI_STOP_SEC_CODE SPI_START_SEC_RAMCODE SPI_STOP_SEC_RAMCODE SPI_START_SEC_CODE_AC SPI_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code (text).
SPI_START_SEC_VAR_NO_INIT_BOOLEAN SPI_STOP_SEC_VAR_NO_INIT_BOOLEAN SPI_START_SEC_VAR_NO_INIT_8 SPI_STOP_SEC_VAR_NO_INIT_8 SPI_START_SEC_VAR_NO_INIT_16 SPI_STOP_SEC_VAR_NO_INIT_16 SPI_START_SEC_VAR_NO_INIT_32 SPI_STOP_SEC_VAR_NO_INIT_32 SPI_START_SEC_VAR_NO_INIT_UNSPECIFIED SPI_STOP_SEC_VAR_NO_INIT_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
SPI_START_SEC_VAR_INIT_BOOLEAN SPI_STOP_SEC_VAR_INIT_BOOLEAN SPI_START_SEC_VAR_INIT_8 SPI_STOP_SEC_VAR_INIT_8	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables

Section Name	Section Type	Description
SPI_START_SEC_VAR_INIT_16 SPI_STOP_SEC_VAR_INIT_16 SPI_START_SEC_VAR_INIT_32 SPI_STOP_SEC_VAR_INIT_32 SPI_START_SEC_VAR_INIT_UNSPECIFIED SPI_STOP_SEC_VAR_INIT_UNSPECIFIED		are initialized by startup code (data).
SPI_START_SEC_VAR_NO_INIT_BOOLEAN_NO_CACHEABLE SPI_STOP_SEC_VAR_NO_INIT_BOOLEAN_NO_CACHEABLE SPI_START_SEC_VAR_NO_INIT_8_NO_CACHEABLE SPI_STOP_SEC_VAR_NO_INIT_8_NO_CACHEABLE SPI_START_SEC_VAR_NO_INIT_16_NO_CACHEABLE SPI_STOP_SEC_VAR_NO_INIT_16_NO_CACHEABLE SPI_START_SEC_VAR_NO_INIT_32_NO_CACHEABLE SPI_STOP_SEC_VAR_NO_INIT_32_NO_CACHEABLE SPI_START_SEC_VAR_NO_INIT_UNSPECIFIED_NO_CACHEABLE SPI_STOP_SEC_VAR_NO_INIT_UNSPECIFIED_NO_CACHEABLE		These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are not cacheable and never cleared and never initialized by startup code (bss).
SPI_START_SEC_VAR_INIT_BOOLEAN_NO_CACHEABLE SPI_STOP_SEC_VAR_INIT_BOOLEAN_NO_CACHEABLE SPI_START_SEC_VAR_INIT_8_NO_CACHEABLE SPI_STOP_SEC_VAR_INIT_8_NO_CACHEABLE SPI_START_SEC_VAR_INIT_16_NO_CACHEABLE SPI_STOP_SEC_VAR_INIT_16_NO_CACHEABLE SPI_START_SEC_VAR_INIT_32_NO_CACHEABLE SPI_STOP_SEC_VAR_INIT_32_NO_CACHEABLE SPI_START_SEC_VAR_INIT_UNSPECIFIED_NO_CACHEABLE SPI_STOP_SEC_VAR_INIT_UNSPECIFIED_NO_CACHEABLE		These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are not cacheable and initialized by startup code (data).
SPI_START_SEC_VAR_NO_INIT_SHAREABLE_NO_CACHEABLE SPI_STOP_SEC_VAR_NO_INIT_SHAREABLE_NO_CACHEABLE		These are all the sections used for variables which are shareable. These variables are not cacheable and never cleared and never initialized by startup code (bss).
SPI_START_SEC_VAR_INIT_SHAREABLE_NO_CACHEABLE SPI_STOP_SEC_VAR_INIT_SHAREABLE_NO_CACHEABLE		These are all the sections which are shareable. These variables are not cacheable and initialized by startup code (data).

Chapter 4 Exclusive Area

SPI module using the services of Schedule Manger (SchM) for entering and exiting critical regions.

The following critical regions are used in the SPI driver:

- Spi.c:
 - Spi_SyncTransmit : exclusive area 1
 - Spi_SyncTransmit : exclusive area 2
 - Spi_ScheduleJobsOnHwUnit : exclusive area 3
 - Spi_AfterOneJobTransferDone : exclusive area 4
 - Spi_WriteIB: exclusive area 4
 - Spi_AsyncTransmit : exclusive area 5
 - Spi_AfterOneJobTransferDone : exclusive area 6
 - Spi_AfterOneJobTransferDone : exclusive area 7

Chapter 5 Interrupt Service Routine (ISR)

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID For FC7300)
FCSPI0	FCSPI0_IRQHandler	94
FCSPI1	FCSPI1_IRQHandler	95
FCSPI2	FCSPI2_IRQHandler	96
FCSPI3	FCSPI3_IRQHandler	97
FCSPI4	FCSPI4_IRQHandler	98
FCSPI5	FCSPI5_IRQHandler	99
FCSPI6	FCSPI6_IRQHandler	146
FCSPI7	FCSPI7_IRQHandler	147

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID For FC7240)
FCSPI0	FCSPI0_IRQHandler	56
FCSPI1	FCSPI1_IRQHandler	57
FCSPI2	FCSPI2_IRQHandler	58
FCSPI3	FCSPI3_IRQHandler	59
FCSPI4	FCSPI4_IRQHandler	60
FCSPI5	FCSPI5_IRQHandler	61

Chapter 6 Error Report

6.1 Det

Function Name	Error Type
Spi_Init	SPI_E_ALREADY_INITIALIZED; SPI_E_PARAM_UNIT; SPI_E_CONFIG_OUT_OF_RANGE; SPI_E_PARAM_CONFIG;
Spi_DeInit	SPI_E_UNINIT; SPI_E_SEQ_PENDING;
Spi_WriteIB	SPI_E_UNINIT; SPI_E_PARAM_CHANNEL; SPI_E_PARAM_CONFIG;
Spi_AsyncTransmit	SPI_E_UNINIT; SPI_E_PARAM_SEQ; SPI_E_SEQ_EMPTY; SPI_E_PARAM_CONFIG; SPI_E_JOB_EMPTY; SPI_E_PARAM_UNIT; SPI_E_PARAM_EB_UNIT; SPI_E_SEQ_PENDING;
Spi_ReadIB	SPI_E_UNINIT; SPI_E_PARAM_CHANNEL; SPI_E_PARAM_CONFIG;
Spi_SetupEB	SPI_E_UNINIT; SPI_E_PARAM_CHANNEL; SPI_E_PARAM_LENGTH; SPI_E_PARAM_CONFIG;
Spi_GetStatus	SPI_E_UNINIT;
Spi_GetJobResult	SPI_E_UNINIT; SPI_E_PARAM_JOB; SPI_E_PARAM_CONFIG;
Spi_GetSequenceResult	SPI_E_UNINIT; SPI_E_PARAM_SEQ; SPI_E_PARAM_CONFIG;
Spi_GetVersionInfo	SPI_E_PARAM_POINTER;
Spi_SyncTransmit	SPI_E_UNINIT; SPI_E_PARAM_SEQ; SPI_E_PARAM_CONFIG; SPI_E_PARAM_UNIT; SPI_E_PARAM_EB_UNIT;

	SPI_E_SEQ_IN_PROCESS;
Spi_GetHWUnitStatus	SPI_E_UNINIT; SPI_E_PARAM_UNIT;
Spi_Cancel	SPI_E_UNINIT; SPI_E_PARAM_SEQ; SPI_E_PARAM_CONFIG;
Spi_SetAsyncMode	SPI_E_UNINIT;
SPI_SetHwAsyncMode	SPI_E_PARAM_UNIT; SPI_E_UNINIT; SPI_E_SEQ_PENDING; SPI_E_PARAM_CONFIG;

6.2 Dem

Function Name	Error Type
Spi_Init	SPI_E_PARAM_UNIT;
Spi_AsyncTransmit	SPI_E_SEQ_PENDING;
Spi_SyncTransmit	SPI_E_SEQ_IN_PROCESS
SPI_SetHwAsyncMode	SPI_E_SEQ_PENDING

Chapter 7 Function Calls

7.1 Function Calls during Startup

The API need be called is `SPI_Init(NULL_Ptr);`

7.2 Function Calls during Shutdown

None

7.3 Function Calls during Wake-up

None

7.4 Function Calls during Runtime

None

Chapter 8 Other Requirements

8.1 Notification, Callback, Callout

- Notification

If the user configures notifications and the value is not "NULL_PTR" or "NULL", an extern declaration will generate in Spi_Cfg.h. The user need implement the notification in any file which includes Spi_Cfg.h.

- Callback

None

- Callout

None

8.2 Macros

Please check various definitions available in Common module's include file Mcal.h for details.

- If OS is not used:

AUTOSAR_OS_NOT_USED need defined.

In this case, If USE_SW_VECTOR_MODE is not defined, the user needs to map the interrupt vector table function to ISR function, for example:

```
extern ISR(FCSPI_0_ISR);

void FCSPi0_IRQHandler(void)
{
    FCSPI_0_ISR();
}
```

- If OS is used:

Do not define AUTOSAR_OS_NOT_USED.

Chapter 9 Integration Steps

- 1) Configure the SPI module and generate configuration files (please refer to Building chapter for details)
- 2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
- 3) Map interrupt notification to their vector locations (please refer to chapter ISR for details).
- 4) Build the SPI module with dependent modules.